
INSIDE CLAUDE CODE

Design Principles of a Powerful Agent based on
Claude Code Source Codes (March 31, 2026)

An Analysis based on Claude Code Source Codes!

<https://github.com/chauncygu/collection-claude-code-source-code>

A collection of the newest Claude Code open source

License and Disclaimer This repository is for academic research and educational purposes only. All subprojects are built from publicly accessible information. Users are responsible for complying with applicable laws, regulations, and service terms.

Source archive of Claude Code and different versions of the clean-room Python rewrite research repository

🔥🔥🔥 News (Pacific Time)

- 12:20 PM · Apr 02, 2026: [Nano Claude Code v3.0: Multi-agent packages, memory package, skill package with built-in skills, argument substitution, fork/inline execution, AI memory search, git worktree isolation, agent type definitions \(~5000 Lines\)](#)
- 7:40 AM · Apr 02, 2026: [Nano Claude Code v2.0: A Minimal Python Reimplementation \(~3400 Lines\), support open and closed source models, skill and memory](#)
- 8:36 AM · Apr 01, 2026: [Nano Claude Code v1.0: A Minimal Python Reimplementation \(~1300 Lines\)](#)
- 0:20 AM · Apr 01, 2026: [Analysis of Claude Code source code \(Video: In Chinese\)](#)
- 19:45 PM · Mar 31, 2026: [Claude Code Research Report \(In Chinese\)](#)
- 17:13 PM · Mar 31, 2026: [Architecture Analysis of Claude Code \(In Chinese\)](#)
- 16:43 PM · Mar 31, 2026: [An Overview of Claude Code Features \(In Chinese\)](#)
- 15:00 PM · Mar 31, 2026: [Hacker News Community Discussion about Claude Code Leak](#)
- 11:48 AM · Mar 31, 2026: [How Anthropic Built 7 Layers of Memory and a Dreaming System for Claude Code](#)
- 9:50 AM · Mar 31, 2026: [Claude code memory analysis](#)



Chaofan Shou

@Fried_rice

Claude code source code has been leaked via a map file in their npm registry!

Code: [...a8527898604c1bbb12468b1581d95e.r2.dev/src.zip](#)

```
shou@dev claude-code % ls -alht cli.js.map
-rw-r--r-- 1 shou staff 57M Mar 31 01:16 cli.js.map
shou@dev claude-code % ls -alht src
total 238k
drwxr-xr-x  8 shou staff 256k Mar 31 01:17 endpoints
-rw-r--r--  1 shou staff 785k Mar 31 01:17 main.tsx
drwxr-xr-x  5 shou staff 1.7K Mar 31 01:17 
drwxr-xr-x 10 shou staff 320k Mar 31 01:17 cli
drwxr-xr-x 103 shou staff 3.2K Mar 31 01:17 commands
drwxr-xr-x 331 shou staff 18k Mar 31 01:17 utils
drwxr-xr-x 144 shou staff 4.4k Mar 31 01:17 components
-rw-r--r--  1 shou staff 44k Mar 31 01:17 QueryEngine.ts
-rw-r--r--  1 shou staff 20k Mar 31 01:17 setup.ts
drwxr-xr-x  5 shou staff 140k Mar 31 01:17 server
drwxr-xr-x 13 shou staff 416k Mar 31 01:17 migrations
drwxr-xr-x 16 shou staff 612k Mar 31 01:17 keybindings
drwxr-xr-x  6 shou staff 192k Mar 31 01:17 skills
drwxr-xr-x  4 shou staff 128k Mar 31 01:17 plugins
-rw-r--r--  1 shou staff 22k Mar 31 01:17 dialogLaunchers.tsx
drwxr-xr-x  5 shou staff 160k Mar 31 01:17 screens
drwxr-xr-x  8 shou staff 2.7k Mar 31 01:17 hooks
-rw-r--r--  1 shou staff 56k Mar 31 01:17 interactiveHelpers.tsx
drwxr-xr-x 38 shou staff 1.2k Mar 31 01:17 services
-rw-r--r--  1 shou staff 3.4k Mar 31 01:17 replLauncher.tsx
drwxr-xr-x  1 shou staff 617k Mar 31 01:17 costHooks.ts
drwxr-xr-x  3 shou staff 96k Mar 31 01:17 monoright
drwxr-xr-x  3 shou staff 96k Mar 31 01:17 assistant
drwxr-xr-x  6 shou staff 192k Mar 31 01:17 remote
drwxr-xr-x  7 shou staff 224k Mar 31 01:17 vlm
drwxr-xr-x  8 shou staff 256k Mar 31 01:17 buddy
drwxr-xr-x 33 shou staff 1.0k Mar 31 01:17 bridge
drwxr-xr-x  8 shou staff 268k Mar 31 01:17 state
drwxr-xr-x 11 shou staff 352k Mar 31 01:17 context
drwxr-xr-x  4 shou staff 128k Mar 31 01:17 upstreamProxy
drwxr-xr-x 23 shou staff 736k Mar 31 01:17 constants
drwxr-xr-x 10 shou staff 328k Mar 31 01:17 types
-rw-r--r--  1 shou staff 2.1k Mar 31 01:17 Task.ts
drwxr-xr-x 10 shou staff 320k Mar 31 01:17 memdir
-rw-r--r--  1 shou staff 25k Mar 31 01:17 commands.ts
drwxr-xr-x  5 shou staff 168k Mar 31 01:17 native-ts
-rw-r--r--  1 shou staff 14k Mar 31 01:17 history.ts
-rw-r--r--  1 shou staff 2.3k Mar 31 01:17 projectOnboardingState.ts
drwxr-xr-x  3 shou staff 96k Mar 31 01:17 outputStyles
-rw-r--r--  1 shou staff 67k Mar 31 01:17 query.ts
drwxr-xr-x  6 shou staff 192k Mar 31 01:17 query
drwxr-xr-x 11 shou staff 352k Mar 31 01:17 tasks
-rw-r--r--  1 shou staff 17k Mar 31 01:17 tools.ts
drwxr-xr-x 45 shou staff 1.4k Mar 31 01:17 tools
drwxr-xr-x  3 shou staff 96k Mar 31 01:17 voice
-rw-r--r--  1 shou staff 1.3k Mar 31 01:17 tasks.ts
-rw-r--r--  1 shou staff 18k Mar 31 01:17 cost-tracker.ts
drwxr-xr-x  3 shou staff 96k Mar 31 01:17 coordinator
-rw-r--r--  1 shou staff 6.3k Mar 31 01:17 context.ts
-rw-r--r--  1 shou staff 3.8k Mar 31 01:17 ink.ts
drwxr-xr-x 58 shou staff 1.6k Mar 31 01:17 ink
-rw-r--r--  1 shou staff 29k Mar 31 01:17 Tool.ts
drwxr-xr-x  3 shou staff 96k Mar 31 01:17 schemas
drwxr-xr-x  3 shou staff 96k Mar 31 01:17 bootstrap
drwxr-xr-x 12 shou staff 384k Mar 31 01:17 ..
```

8:23 AM · Mar 31, 2026 · 35.7M Views

Overview: The Agent Harness

Claude Code is an **Agent Harness** surrounding the model:
a loop, a policy layer, a tool system, a memory system, and a context-management engine wrapped around a frontier model.

The intelligence of the system emerges from the agentic architecture, not solely from the underlying LLM.

It provides the continuity, safety control, and tools necessary to turn intelligence into action.

FOUR-LAYER ARCHITECTURE

1 Engine Layer The core control loop and model interaction

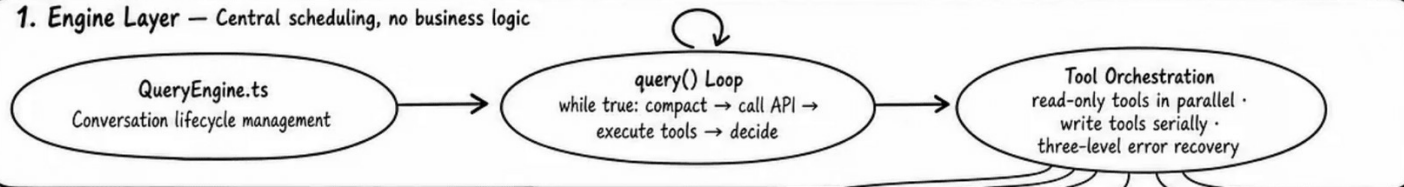
2 Tool Layer Execution environments and filesystem access

3 Service Layer External integrations via MCP

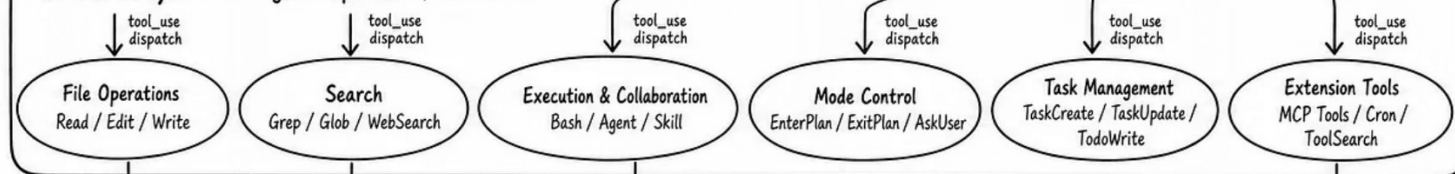
4 Safety & Governance Permissions, hooks, and bash security

Claude Code Four-Layer Architecture

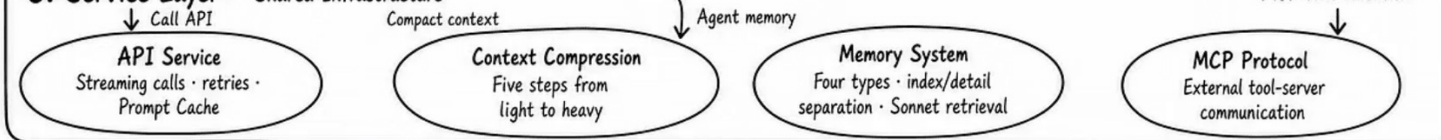
1. Engine Layer — Central scheduling, no business logic



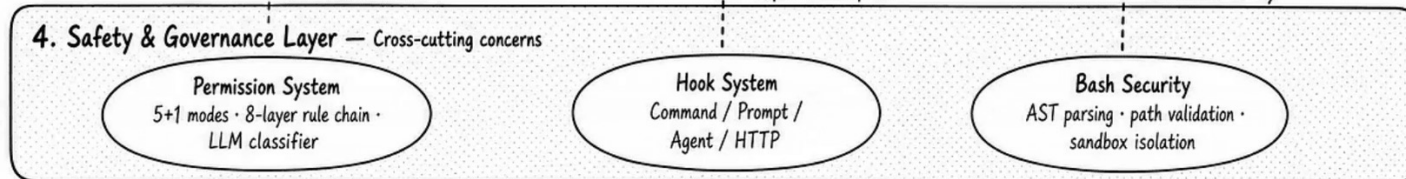
2. Tool Layer — All Agent capabilities, 40+ tools



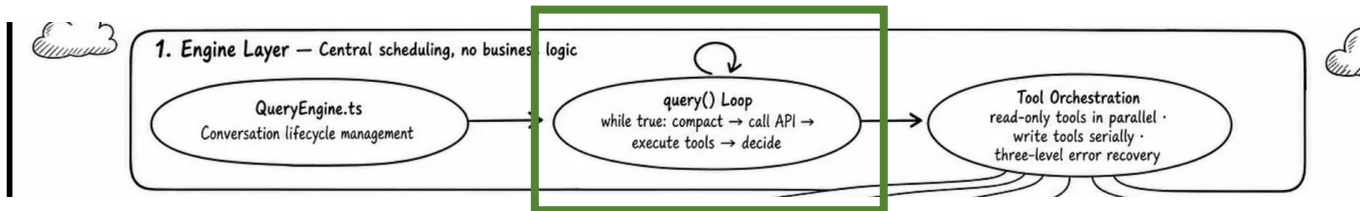
3. Service Layer — Shared Infrastructure



4. Safety & Governance Layer — Cross-cutting concerns



A REPEATING TOOL-USE LOOP



1 Propose Action

The model analyzes the current context and decides which tool to call to advance the goal.

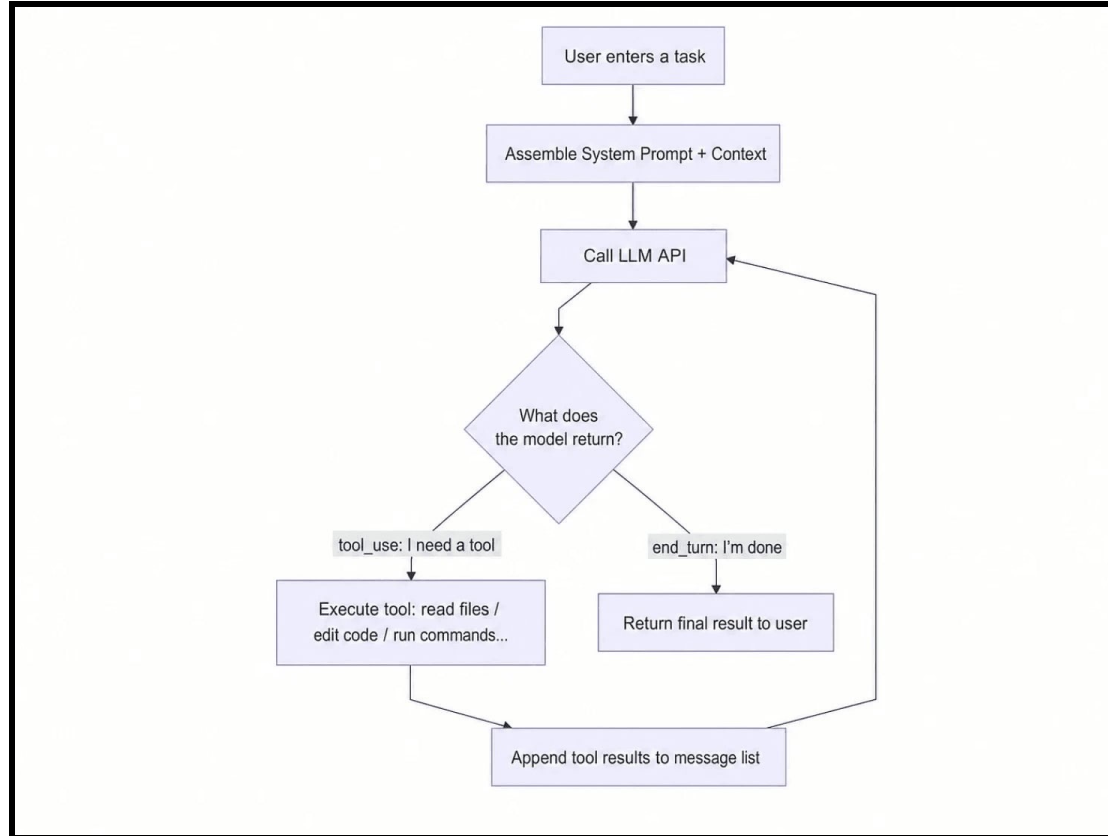
2 Execute Tool

The runtime executes the requested tool in the bounded environment, enforcing safety and permissions.

3 Observe Results

The output, errors, or state changes are captured and appended back to the context window to inform the next cycle.

A REPEATING TOOL-USE LOOP



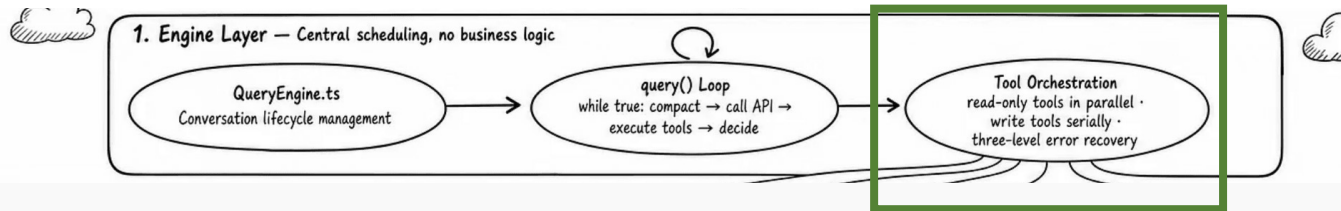
A REPEATING TOOL-USE LOOP

Verification is what closes the loop. An action is not complete when the code is written; it is complete when the **test passes**.

File contents, linter output, and shell command results become fresh evidence in the context window.

A reliable agent **does not guess** whether a change succeeded; it measures.

PARALLELIZE OBSERVATION, SERIALIZE MUTATION



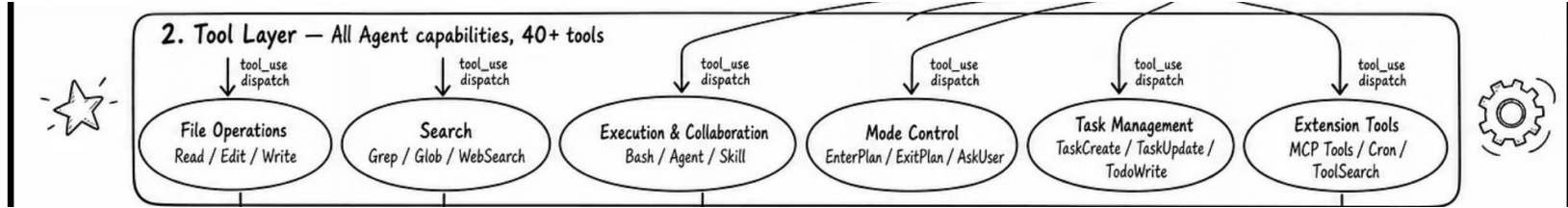
PARALLEL OBSERVATION (read-only)

Reading multiple files, searching the codebase, and retrieving memory are safe to execute concurrently. They gather context quickly without risk of conflict.

SERIAL MUTATION (write)

Editing files, executing shell commands, and committing changes must be done **one step at a time** to ensure the environment state remains predictable.

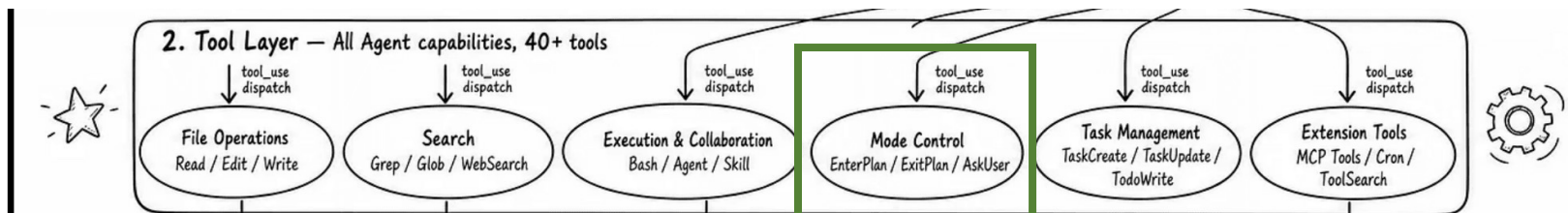
SPECIALIZED TOOLS REDUCE AMBIGUITY



The agent should use specialized tools instead of generic Bash when possible. Specialized tools fail predictably and return **actionable errors**, whereas a generic bash execution might fail silently or catastrophically:

- a Read tool instead of `cat src/index.ts`
- an Edit tool instead of `sed -l`
- a Write tool instead of `echo > file`
- a Glob tool instead of `find`
- a Grep tool instead of raw `grep` or `rg`

Plan Mode



`/plan` refactor auth without changing public APIs

- 01 **Explore before acting.** The agent inspects the repository, constraints, and affected surfaces without changing state.
- 02 **Externalize the intended approach.** The plan makes scope, sequence, and assumptions visible to the developer.
- 03 **Require an explicit transition.** Implementation begins only after the plan has created a clear boundary between reasoning and execution.
- 04 **Reduce costly reversals.** Misalignment is corrected before edits, commands, or repository mutations accumulate.

Plan Mode Changes Permissions

- **Read-oriented work remains available.** Claude can inspect files, search the codebase, and gather evidence needed to form a plan.
- **State-changing actions are constrained.** File edits, risky commands, and other mutations wait until the developer approves the approach.
- **The interface communicates phase.** The user can distinguish investigation from implementation instead of inferring intent from individual tool calls.

Safety emerges from separating what the agent may understand now from what it may change next.

When to Use Plan Mode

WHEN TO USE PLANNING

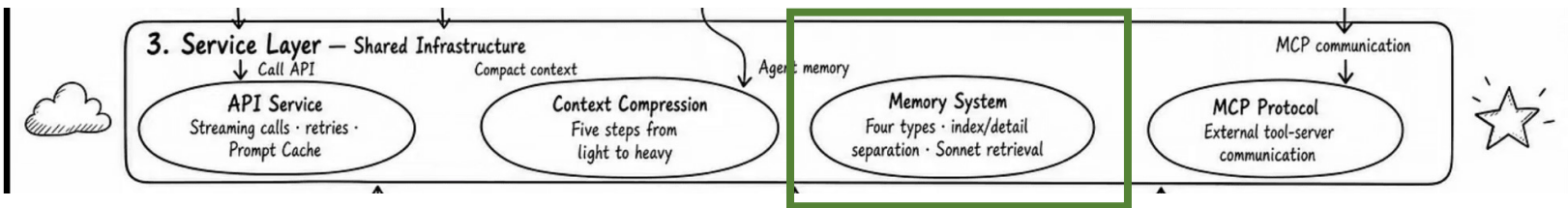
Ambiguous architecture, multi-file changes, migrations, unfamiliar subsystems, and high-impact work.

WHEN TO SKIP IT

Obvious one-line fixes, mechanical renames, and changes where the intended diff is already clear.

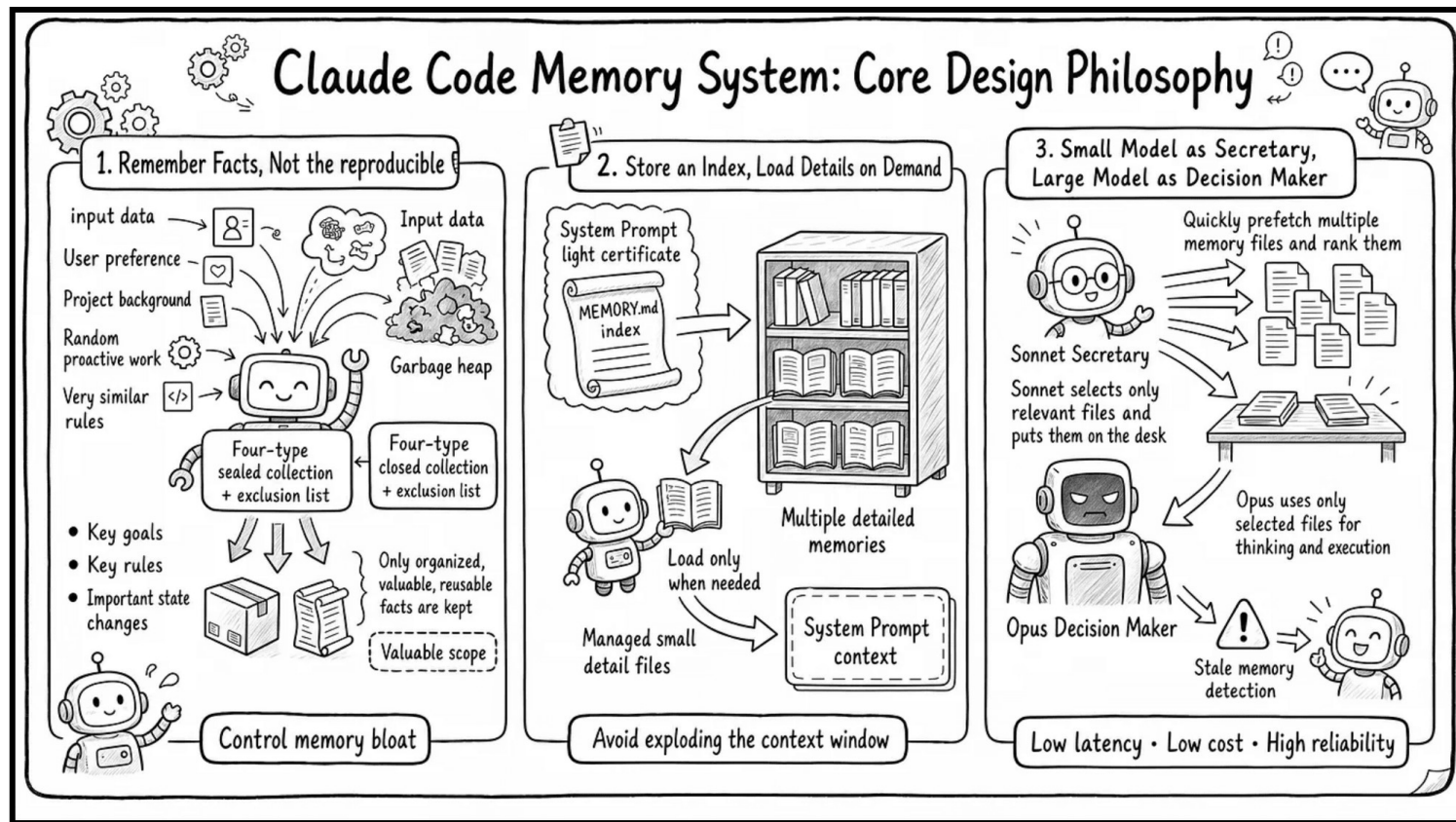
The design principle is adaptive process: match deliberation cost to uncertainty and consequence.

MEMORY PRESERVES DECISIONS



- The memory system is not designed to perfectly reproduce past inputs and outputs.
- Instead, it saves the "**why**"—the architectural decisions, the hard-learned constraints, and the project-specific rules.
- By discarding the raw transcript and keeping the insights, memory acts as a **distilled policy engine** (what decision to make given context) rather than a search index.

WHAT THE MEMORY SYSTEM KEEPS



TWO MEMORY SYSTEMS SERVE DIFFERENT OWNERS

CLAUDE.MD

User-authored, Explicit, and global. It contains the non-negotiable rules, architectural constraints, and style guides dictated by the human developer.

AUTO MEMORY (memory.md)

Agent-authored, implicit, and learned. It stores the operational history, discovered workarounds, and implicit context that Claude learns by working in the repository.

Together, they form a complete policy: what the human demands, and what the agent discovers.

MEMORY INDEXING AND ON-DEMAND LOADING

Claude Code Memory Files and Indexing

1. A Memory File Stores Structured Feedback

```
---  
name: no-mock-database  
description: integration tests must  
use the real database, not mocks  
type: feedback  
---
```

Integration tests must use the real database, not mocks.

****Why:**** Last quarter, mock-based tests passed, but the production migration failed.

****How to apply:**** When writing tests in this module, always connect to the real database.

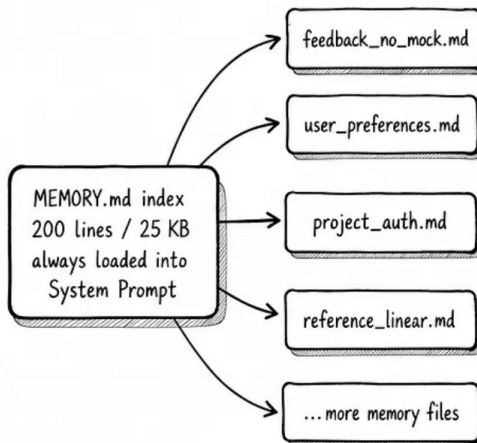
Typed memory = behavior instruction,
not raw chat history

2. MEMORY.md Keeps a Lightweight Index

```
ooo  
- [No Mock Database]  
  (feedback_no_mock_db.md)  
  — tests must use real DB  
- [User Preferences]  
  (user_preferences.md)  
  — prefers terse responses  
- [Auth Rewrite]  
  (project_auth_rewrite.md)  
  — driven by compliance,  
  not tech debt
```

The system prompt sees the index first,
not every full memory.

3. Details Live in Separate Files



Load details only when relevant
to avoid context bloat.

MEMORY FROM CLAUDE CLI

INITIALIZE AND INSPECT MEMORY

```
/init  
/memory
```

Create a starter CLAUDE.md, then list and edit the memory files loaded in the current session.

WRITE AUTO-MEMORY

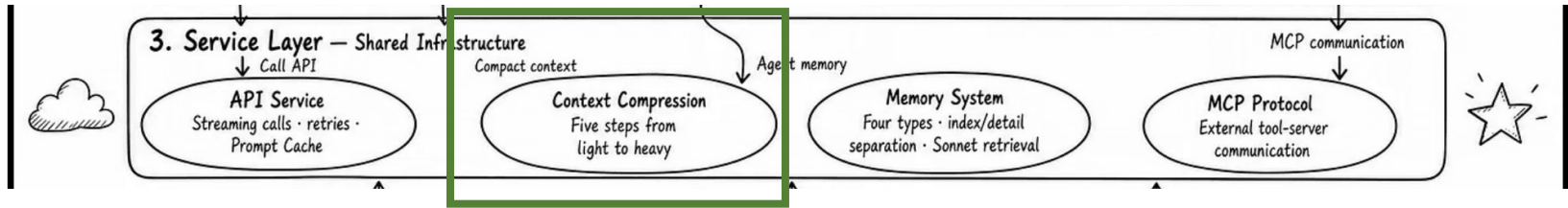
```
Remember that integration tests require a local Redis instance.
```

Claude can retain durable corrections and preferences in auto-memory when it is enabled.

WRITE OR LOOK UP PROJECT RULES

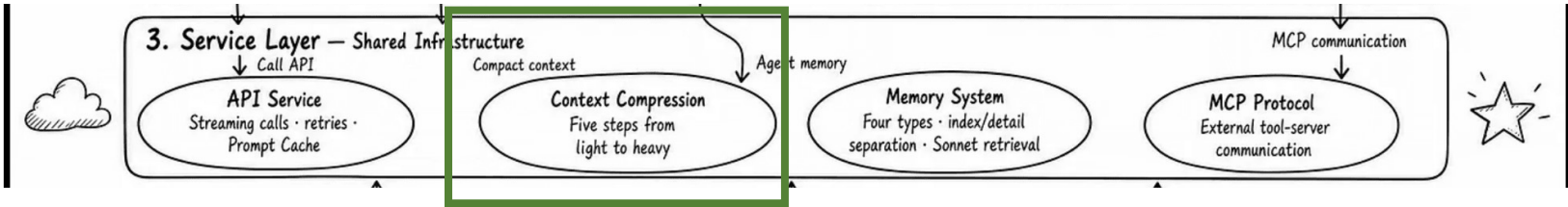
```
Add this to CLAUDE.md: use pnpm, not npm.  
Check project memory for the integration-test prerequisites.
```

Context Compression



- Every action, observation, and error in the control loop consumes tokens. The context window fills rapidly.
- As the window approaches its limit, **attention degrades**, latency spikes, and the model begins to forget earlier constraints.
- Because an agent is a continuous process, **active context management** is required to keep it functioning over long sessions.

Context Compression



- When the context nears a predefined token threshold, server-side **compaction** is triggered automatically.
- Claude Code uses a **portfolio of mechanisms**, including tool-level previews (truncating long file reads) and message auto-compaction.
- It uses an LLM to summarize earlier turns, replacing verbose history with a dense summary while **preserving the system prompt**.
- Recent operations are kept fully intact, ensuring the agent doesn't lose track of its immediate task state.

COMPACTION MECHANISMS

Mechanism	What it does	Why it matters
Large result persistence	Stores large tool outputs outside the prompt and returns a small preview	Prevents one command from consuming the session
Preview generation	Shows a compact summary, such as a 2 KB preview of a larger result	Keeps the model informed without flooding context
Snip or auto-compact	Removes or summarizes older conversation spans when token pressure rises	Preserves continuity across long sessions
Micro-compact	Clears old compactable tool results while keeping recent ones	Reduces repeated tool-result baggage
Context collapse	Applies compaction before API calls when context is near limits	Avoids hitting the context wall mid-task
Reserved token margin	Leaves room for the model's next output and tool requests	Prevents the agent from using every token on input
Post-compact restoration	Restores a small number of important files, plan state, or skills after compaction	Helps the agent regain working memory after compression

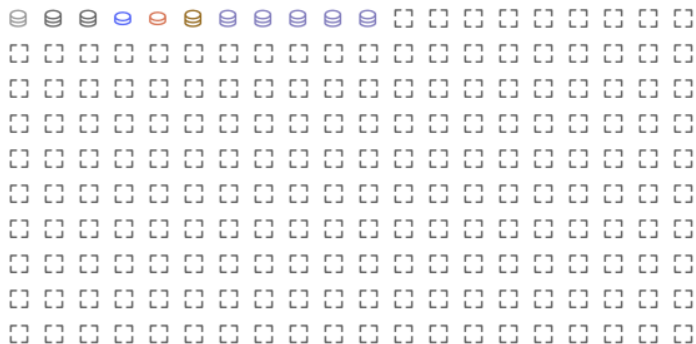
Aways Inspect your Context

INSPECT, COMPACT, AND CONFIGURE

/context	Inspect what is filling the context window.
/compact	Preserve open tasks, changed files, test status, and next steps.
/config autoCompact=true false	Set automatic compaction.

> /context

Context Usage

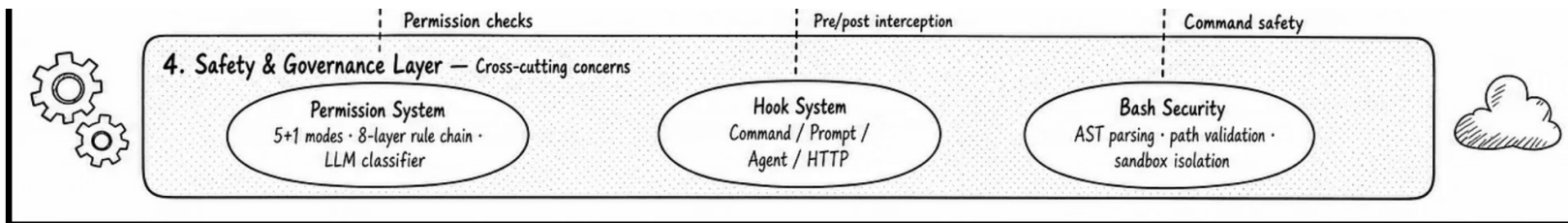


Opus 4.6 (1M context)
global.anthropic.claude-opus-4-6-v1[1m]
48.7k/1m tokens (5%)

Estimated usage by category

System prompt: 6.3k tokens (0.6%)
System tools: 11.4k tokens (1.1%)
Custom agents: 164 tokens (0.0%)
Memory files: 2.6k tokens (0.3%)
Skills: 4.4k tokens (0.4%)
Messages: 23.8k tokens (2.4%)
Free space: 951.3k (95.1%)

Safety and Governance



1. **Prompt Level:** The system prompt explicitly defines safety boundaries, Git caution, and operational rules before execution begins.
2. **Tool Level:** Strict schemas force the model to declare its intent clearly, preventing ambiguous or excessively broad operations.
3. **Execution Level:** The runtime intercepts risky bash commands and requires human approval before applying state changes.

System Prompt: A Safety Policy Stack

STABLE LAYERS

Define the agent's role, safety boundaries, operational caution, tool use schemas, Git safety rules, and output style.

DYNAMIC LAYERS

Inject current environment data, project-specific instructions, memory guidance, and connected MCP services.

This turns the prompt into a runtime-composed policy surface rather than a single static block of prose.

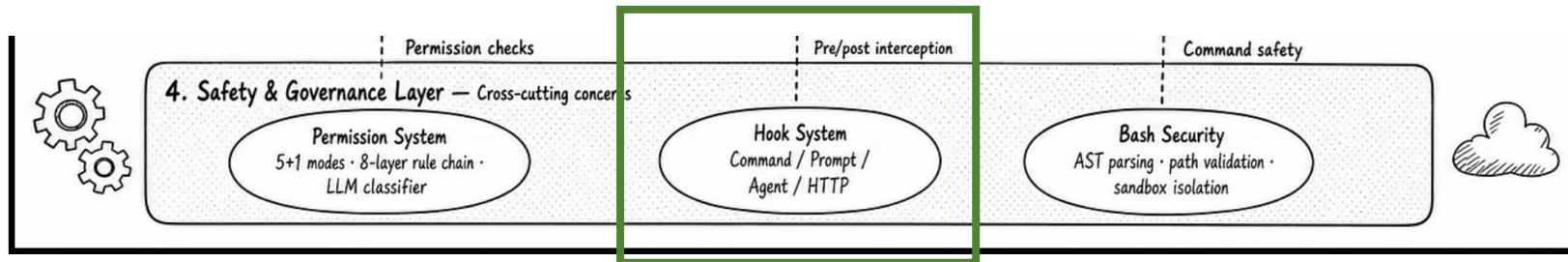
STABLE AND DYNAMIC PROMPT LAYERS

[Role Definition] You are an interactive agent, helping users complete...	← Same for all users
[Safety Boundary] Important: authorized security testing is allowed...	← Same for all users
[Behavior Rules] Generally, do not modify code you have not read...	← Same for all users
[Operation Safety] Carefully consider reversibility...	← Same for all users
[Tool Usage] When dedicated tools are available...	← Same for all users
[Git Safety] Never modify git config...	← Same for all users
[Output Style] Get straight to the point; be extremely concise...	← Same for all users
----- __SYSTEM_PROMPT_DYNAMIC_BOUNDARY__ -----	
[Environment Info] Working directory: /Users/you/my-project	← Different for each user
[CLAUDE.md] This project uses TypeScript + Jest...	← Different for each project
[Memory Instructions] You have a persistent memory system...	← May differ each conversation
[MCP Instructions] You are connected to GitHub MCP server...	← Different for each user

Behavior Rules

- **Read code before modification.** The agent should not modify files it has not inspected.
For example, before editing `src/auth.ts`, it should first read the file and understand the existing login flow
- **“Less is More” coding style.** Claude Code follows a “less is more” coding style.
For example, if a bug is caused by a missing null check, the agent should add that check only; it should not refactor the whole module, introduce a new helper class, or redesign the error-handling system unless the user asks.
- **Diagnose failure before changing strategy.** When a command or fix fails, the agent should read the error message and test its assumptions before trying something else.
For example, if `npm test` fails because a mock database is unavailable, the agent should inspect the test setup and configuration instead of blindly rerunning the same command or switching to an unrelated fix.

HOOKS: DETERMINISTIC AUTOMATION



Hooks execute user-defined shell commands at specific events. They provide **deterministic control** over behavior, enforcing rules automatically rather than relying on the LLM's judgment:

1. Session Lifecycle

SessionStart: Fires when a session begins or resumes.

Setup: Fires when starting Claude Code with `--init-only`, `--init`, or `--maintenance` in `-p` mode.

SessionEnd: Fires when a session terminates.

2. User Input & Prompts

UserPromptSubmit, *UserPromptExpansion*, *InstructionsLoaded*: Handle input, expansion, and context loading (e.g., `CLAUDE.md`).

3. Tool Control & Guardrails (Agentic Loop)

Includes *PreToolUse*, *PermissionRequest*, *PermissionDenied*, *PostToolUse*, *PostToolUseFailure*, and *PostToolBatch* to manage tool execution, permissions, and outcomes, including blocking actions for the AI agent.

CONFIGURING AND VERIFYING HOOKS

1. DEFINE IN .CLAUDE/SETTINGS.JSON

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [{
          "type": "command",
          "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write"
        }]
      }
    ]
  }
}
```

2. VERIFY AND INSPECT IN SESSION

/hooks	Opens the read-only hooks browser to confirm registration
--------	---

SEVEN PRINCIPLES FOR BUILDING POWERFUL AGENTS

- 1 The Agent Harness Is the Product
- 2 Architecture Is a Set of Contracts
- 3 The Agent Is a Repeating Tool-Use Loop with Verification
- 4 Plan Mode Creates a Permission Boundary
- 5 The System Prompt Is a Policy Stack for Safety and Governance
- 6 Memory Preserves Decisions, Not Transcripts
- 7 Context Is the Agent's Scarcest Resource

Thank you

Subscribe my latest presentations @
YanAITalk Youtube Channel



YanAITalk

@yanaitalk • 4.55K subscribers • 90 videos

Get empowered with AI! ...more